
Decouverte du tenant courant dans la plateforme comptabilite

Flux `discover-contexts` / `select-context` et injection `X-Tenant-Id`

Comment le front Next.js (`accounting.yowyob.com`) determine **le tenant courant** d'un utilisateur : decouverte des contextes cote Kernel, selection du contexte/organisation, persistance en `localStorage`, puis re-injection de l'en-tete `X-Tenant-Id` sur chaque appel API.

Auteur : ALD • `azangueleonel9@gmail.com`

Composant	<code>FRONTEND/components/landing/login-modal.tsx</code>
Clients API	<code>src/lib</code> (Kernel) • <code>src/lib2</code> (backend accounting)
Principe	le tenant est decouvert cote serveur, jamais code en dur
Date	27 juin 2026

Table des matières

1	Vue d'ensemble	2
2	Le flux en trois temps	2
2.1	Etape 1 — Decouverte (<code>discover-contexts</code>)	2
2.2	Etape 2 — Selection (<code>select-context</code>)	2
2.3	Etape 3 — Persistance (<code>localStorage</code>)	3
3	Re-injection dans les appels suivants	3
4	Etat du code : bug corrige	4
5	Resume	4

1. Vue d'ensemble

Dans la plateforme comptabilite, **le tenant courant n'est pas saisi ni devine cote navigateur** : il est *decouvert* aupres du Kernel a partir du seul couple e-mail / mot de passe, puis **choisi** (quand le compte appartient a plusieurs tenants ou organisations), **persiste** en `localStorage`, et enfin **re-injecte** comme en-tete `X-Tenant-Id` sur tous les appels suivants.

Principe directeur

Le navigateur n'envoie **ni tenant ni cle d'API** a l'etape de decouverte. C'est le **kernel-core** qui resout lui-meme l'ensemble des tenants associes au principal — c'est ce qui rend le multi-tenant reel (un meme e-mail peut exister dans plusieurs tenants).

2. Le flux en trois temps

2.1 Etape 1 — Decouverte (`discover-contexts`)

Au login, `handleLogin` appelle le backend accounting (qui detient les cles Kernel cote serveur) **sans** tenant ni cle :

```
POST {accounting-backend}/api/auth/discover-contexts
Content-Type: application/json

{ "email": "...", "password": "..." }
```

La reponse contient un `selectionToken` et la liste des `contexts`. Chaque contexte est de la forme :

```
{
  "contextId": "...",
  "tenantId": "...",
  "organizations": [ { "organizationId": "...", ... }, ... ]
}
```

Vrai multi-tenant

Le Kernel renvoie **tous** les contextes du principal. Un meme e-mail present dans plusieurs tenants donne donc plusieurs contextes : c'est le serveur qui decouvre l'appartenance, pas le front.

2.2 Etape 2 — Selection (`select-context`)

La fonction `buildOptions` aplatit les contextes en options selectionnables (une par organisation, ou une seule pour un contexte sans organisation). Deux cas :

- **une seule option** : `completeSelection(...)` est appelee automatiquement (pas d'ecran de choix) ;
- **plusieurs options** : l'utilisateur choisit son espace (multi-tenant / multi-org).

```
POST {accounting-backend}/api/auth/select-context
Content-Type: application/json

{
  "selectionToken": "...",
  "contextId": "<contexte choisi>",
  "organizationId": "<organisation choisie>"
}
```

La reponse fournit le **token final** et le **tenantId** du contexte retenu.

2.3 Etape 3 — Persistance (localStorage)

```
localStorage.setItem('auth_token', response.token);
localStorage.setItem('user', JSON.stringify(response.user));

// Tenant/organisation issus du CONTEXTE choisi, repli sur l'env
// uniquement en dernier recours (deploiement mono-client).
localStorage.setItem('tenant_id',
  response.tenantId || option.tenantId
  || process.env.NEXT_PUBLIC_TENANT_ID || '');
localStorage.setItem('organization_id',
  option.organizationId || response.user?.organizationId
  || process.env.NEXT_PUBLIC_ORGANIZATION_ID || '');

OpenAPI.TOKEN = response.token;
```

Le tenant courant = le tenantId du contexte choisi, stocke dans
localStorage.tenant_id.

3. Re-injection dans les appels suivants

Les deux clients API generes relisent localStorage a chaque requete et posent l'en-tete tenant :

Client	Fichier	En-tete + repli
Kernel	src/lib/core/OpenAPI.ts	X-Tenant-ID tenant_id → NEXT_PUBLIC_TENANT_ID
Backend accounting	src/lib2/core/OpenAPI.ts	X-Tenant-Id (+ X-Organization-Id) tenant_id → NEXT_PUBLIC_TENANT_ID → 11111111...

```
// src/lib2/core/OpenAPI.ts (client backend accounting)
'X-Organization-Id': localStorage.getItem('organization_id')
  || process.env.NEXT_PUBLIC_ORGANIZATION_ID
  || '4e177ff2-89b8-4d24-926a-5763dfa1b19a',
'X-Tenant-Id': localStorage.getItem('tenant_id')
  || process.env.NEXT_PUBLIC_TENANT_ID
  || '11111111-1111-1111-1111-111111111111',
```

Token vs en-tete

Le token porte deja le tenant dans son claim tid. L'en-tete X-Tenant-Id est redondant mais reste requis par les filtres; cote backend accounting, app.organization.require-explicit=true impose en plus un X-Organization-Id non vide (sinon « no organization context »).

4. Etat du code : bug corrige

Version corrige

Le code actuel est la **version corrige** du bug historique « tenant code en dur ». Auparavant, `login-modal.tsx` envoyait `NEXT_PUBLIC_TENANT_ID` (souvent `11111111...`, le tenant *plateforme*) dans le corps du login, provoquant des 401 puis un throttle Kernel. Desormais le front **decouvre** les contextes via `discover-contexts` / `select-context` et envoie le *vrai* tenant ; `NEXT_PUBLIC_TENANT_ID` n'est plus qu'un dernier recours pour un deploiement mono-client.

Risque residuel

Si `localStorage.tenant_id` et `NEXT_PUBLIC_TENANT_ID` sont vides, le client `src/lib2` retombe sur le tenant plateforme code en dur `11111111-1111-1111-1111-111111111111`, et `X-Organization-Id` sur l'org seed `4e177ff2...`. Un appel partirait alors avec le mauvais scope. A surveiller en cas de session sans login prealable.

5. Resume

1. **Decouverte** : `POST /api/auth/discover-contexts` avec `{email, password}` seulement → le Kernel renvoie tous les contextes (tenants + organisations) + un `selectionToken`.
2. **Selection** : auto si une seule option, sinon choix utilisateur, via `POST /api/auth/select-context` → token final + `tenantId`.
3. **Persistence** : `tenant_id`, `organization_id`, `auth_token` en `localStorage`.
4. **Reutilisation** : les clients `src/lib` et `src/lib2` injectent `X-Tenant-Id` (et `X-Organization-Id`) depuis `localStorage` a chaque appel.

Le tenant courant est donc decouvert cote serveur, choisi par l'utilisateur, puis transporte de bout en bout via le `localStorage` et l'en-tete `X-Tenant-Id`.